

Кодирование алгоритма логистической регрессии

В этом разделе мы узнаем, как вручную закодировать алгоритм логистической регрессии. Код для этого раздела приведен далее.

- **Блокнот** Coding_logistic_regression.ipynb:
https://github.com/luisguiserrano/manning/blob/master/Chapter_6_Logistic_Regression/Coding_logistic_regression.ipynb.

Мы протестируем код на том же наборе данных, который применяли в главе 5. Набор данных приведен в табл. 6.9.

Таблица 6.9. Набор данных, который будем использовать с логистическим классификатором

Аак x_1	Бип x_2	Метка y
1	0	0
0	2	0
1	1	0
1	2	0
1	3	1
2	2	1
2	3	1
3	2	1

Далее приведен код для загрузки нашего небольшого набора данных, а его график показан на рис. 6.8:

```
import numpy as np
features = np.array([[1,0],[0,2],[1,1],[1,2],[1,3],[2,2],[2,3],[3,2]])
labels = np.array([0,0,0,0,1,1,1,1])
```

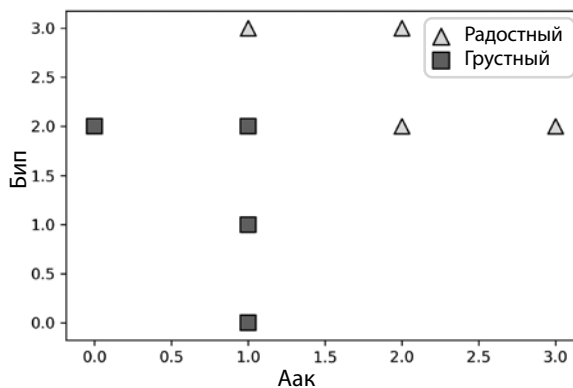


Рис. 6.8. График набора данных, где радостные предложения представлены треугольниками, а грустные — квадратами

Кодирование алгоритма логистической регрессии вручную

В этом разделе мы увидим, как вручную закодировать логистический метод и алгоритм логистической регрессии. Обобщая, будем кодировать алгоритм логистической регрессии для набора данных с n весами. Используемые обозначения:

- признаки — $x_1, x_2 \dots x_n$;
- метка — y ;
- веса — $w_1, w_2 \dots w_n$;
- смещение — b .

Оценка для конкретного предложения представляет собой сигмоидную функцию суммы весов каждого слова (w_i), умноженной на количество появлений (x_i), плюс смещение (b). Обратите внимание на то, что мы используем обозначение суммирования для

$$\sum_{i=1}^n a_i = a_1 + a_2 + \dots + a_n.$$

$$\text{Прогноз: } \hat{y} = \sigma(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b) = \sigma\left(\sum_{i=1}^n w_i x_i + b\right).$$

Рассматривая проблему, мы будем ссылаться на $x_{\text{аак}}$ и $x_{\text{бип}}$ как на x_1 и x_2 соответственно. Их веса равны w_1 и w_2 , а смещение — b .

Начинаем с кодирования сигмоидной функции, оценки и прогноза. Напомним, что формула сигмоидной функции выглядит так:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

```
def sigmoid(x):
    return np.exp(x)/(1+np.exp(x))
```

Для функции оценки мы используем скалярное произведение между признаками и весами. Напомним, что скалярное произведение векторов $(x_1, x_2 \dots x_n)$ и $(w_1, w_2 \dots w_n)$ равно $w_1 x_1 + w_2 x_2 + \dots + w_n x_n$.

```
def score(weights, bias, features):
    return np.dot(weights, features) + bias
```

Наконец, вспомним, что прогноз — это сигмоидная функция активации, применяемая к оценке:

```
def prediction(weights, bias, features):
    return sigmoid(score(weights, bias, features))
```

Теперь, когда у нас есть прогноз, можем перейти к логарифмической потере. Вспомним формулу логарифмической потери:

$$\text{логарифмическая потеря} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}).$$

Закодируем ее следующим образом:

```
def log_loss(weights, bias, features, label):
    pred = prediction(weights, bias, features)
    return -label*np.log(pred) - (1-label)*np.log(1-pred)
```

Нам нужна логарифмическая потеря по всему набору данных, поэтому сложим их для всех точек данных, как показано далее:

```
def total_log_loss(weights, bias, features, labels):
    total_error = 0
    for i in range(len(features)):
        total_error += log_loss(weights, bias, features[i], labels[i])
    return total_error
```

Теперь мы готовы к кодированию метода логистической регрессии и алгоритма логистической регрессии. Вспомним, что для более чем двух переменных шаг логистической регрессии для i -го веса представляет собой следующую формулу, где η — скорость обучения:

$$w_i \rightarrow w_i + \eta(y - \hat{y})x_i, \text{ где } i = 1, 2 \dots n;$$

$$b \rightarrow b + \eta(y - \hat{y}), \text{ где } i = 1, 2 \dots n.$$

```
def logistic_trick(weights, bias, features, label, learning_rate = 0.01):
    pred = prediction(weights, bias, features)
    for i in range(len(weights)):
        weights[i] += (label-pred)*features[i]*learning_rate
        bias += (label-pred)*learning_rate
    return weights, bias
```

```
def logistic_regression_algorithm(features, labels, learning_rate = 0.01,
    epochs = 1000):
    utils.plot_points(features, labels)
    weights = [1.0 for i in range(len(features[0]))]
    bias = 0.0
    errors = []
    for i in range(epochs):
        errors.append(total_log_loss(weights, bias, features, labels))
        j = random.randint(0, len(features)-1)
        weights, bias = logistic_trick(weights, bias, features[j],
            labels[j])
    return weights, bias
```

Теперь мы можем запустить алгоритм логистической регрессии для построения логистического классификатора, который соответствует нашему набору данных:

```
logistic_regression_algorithm(features, labels)
([0.46999999999999953, 0.09999999999999937], -0.6800000000000004)
```

Полученный нами классификатор имеет следующие веса и смещение:

- $w_1 = 0,47$;
- $w_2 = 0,1$;
- $b = -0,68$.

График классификатора (вместе с графиком предыдущих классификаторов в каждый из периодов) показан на рис. 6.9.

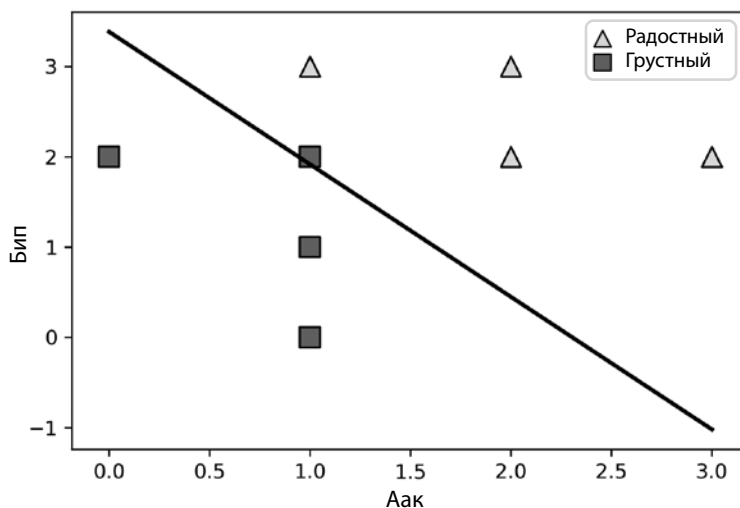


Рис. 6.9. Граница результирующего логистического классификатора

На рис. 6.10 мы видим график классификаторов, соответствующих всем периодам (*слева*), и график логарифмических потерь (*справа*). На графике промежуточных классификаторов (см. рис. 6.10, *слева*) конечный классификатор соответствует жирной линии. Обратите внимание на график логарифмических потерь. По мере запуска алгоритма для большего количества периодов логарифмическая потеря резко уменьшается, а это именно то, чего мы добиваемся. Кроме того, логарифмическая ошибка никогда не равна нулю, даже если все точки классифицированы правильно. Это связано с тем, что для любой точки независимо от того, насколько хорошо она классифицирована, логарифмическая ошибка никогда не равна нулю. Сравним это с рис. 5.26, где потери персептрона достигают нулевого значения, когда каждая точка классифицирована правильно.

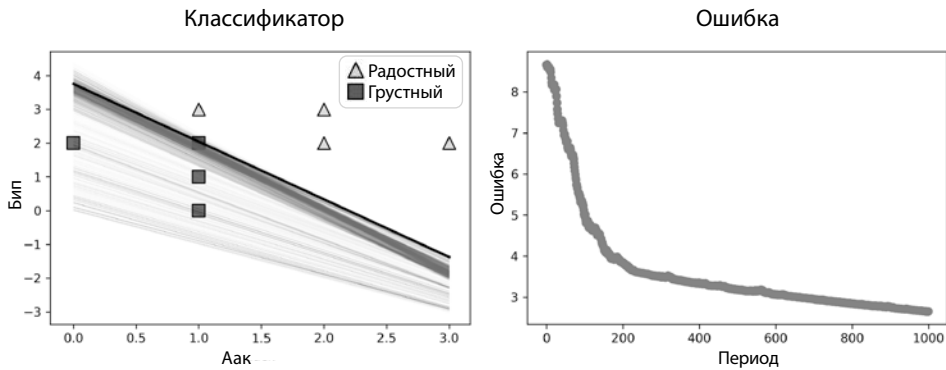


Рис. 6.10. Графики: *слева* — всех промежуточных шагов алгоритма логистической регрессии; *справа* — ошибок

Реальное применение. Классификация обзоров IMDB с помощью Turi Create

В данном разделе мы увидим реальное применение логистического классификатора при анализе настроений. Задействуем Turi Create для построения модели, которая анализирует обзоры фильмов на популярном сайте IMDB. Код для этого раздела приведен далее.

- **Блокнот** `Sentiment_analysis_IMDB.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_6_Logistic_Regression/Sentiment_analysis_IMDB.ipynb
- **Набор данных** `IMDB_Dataset.csv`.

Сначала импортируем Turi Create, загрузим набор данных и преобразуем его в SFrame, который назовем `movies`:

```
import turicreate as tc
movies = tc.SFrame('IMDB Dataset.csv')
```

Первые пять строк набора данных приведены в табл. 6.10. В графе **Отзыв** содержится текст отзыва, а в графе **Настроение** — настроение отзыва, которое может быть положительным или отрицательным.

Таблица 6.10. Первые пять строк набора данных IMDB

Отзыв	Настроение
One of the other reviewers has mentioned... (Один из других рецензентов отметил...)	Положительный
A wonderful little production... (Замечательная постановка...)	Положительный
I thought this was a wonderful day to spend... (Я подумал, сегодня прекрасный день для того, чтобы провести...)	Положительный
Basically, there's a family where a little... (В общем, есть семья, где маленький...)	Отрицательный
Petter Mattei's "Love in the time of money" is a... («Любовь во времена денег» Питера Маттеи — это...)	Положительный

Набор данных состоит из двух граф: одна с обзором в виде строки, вторая с настроением — положительным или отрицательным. В первую очередь обрабатываем строку, потому что каждое слово должно стать отдельным признаком. Для этой задачи пригодится встроенная функция `count_words` Turi Create в пакете `text_analytics`, поскольку она превращает предложение в словарь с подсчетом слов. Например, предложение «быть или не быть» превращается в словарь {'быть':2, 'или':1, 'не':1}. Затем добавим еще одну графу под названием `words`, содержащую этот словарь:

```
movies['words'] = tc.text_analytics.count_words(movies['review'])
```

В табл. 6.11 показаны первые несколько строк набора данных с новой графой. В ней размещены признаки для нашего логистического классификатора.

Таблица 6.11. Графа «Слова» представляет собой словарь, в который записывается каждое слово в обзоре и количество его появлений

Отзыв	Настроение	Слова
One of the other reviewers has mentioned...	Положительный	{'if': 1.0, 'viewing': 1.0, 'comfortable': 1.0, ...}
A wonderful little production...	Положительный	{'done': 1.0, 'every': 1.0, 'decorating': 1.0, ...}
I thought this was a wonderful day to spend...	Положительный	{'see': 1.0, 'go': 1.0, 'great': 1.0, 'superm ...}
Basically, there's a family where a little...	Отрицательный	{'them': 1.0, 'ignore': 1.0, 'dialogs': 1.0, ...}
Peter Mattei's "Love in the Time of Money" is a...	Положительный	{'work': 1.0, 'his': 1.0, 'for': 1.0, 'anxiously': ...}

Мы готовы обучать свою модель! Для этого используем функцию `create` в пакете `logistic_classifier`, в которой целью (меткой) будет графа `sentiment`, а признаками — графа `words`. Обратите внимание на то, что цель выражена в виде строки с именем графы, содержащей метку, а признаки имеют вид массива строк с именами граф, содержащих каждый из признаков (если нужно указать несколько граф):

```
model = tc.logistic_classifier.create(movies, features=['words'],
target='sentiment')
```

Теперь, когда мы обучили модель, можем посмотреть на веса слов с помощью команды `coefficients`. В полученной таблице несколько граф, но нас интересуют `index` и `value`, которые показывают слова и их веса. Первая пятерка:

- (intercept) — 0,065;
- if — -0,018;
- viewing — 0,089;
- comfortable — 0,517;
- become — 0,106.

Первая названная графа `intercept` — это смещение. Поскольку смещение модели положительное, то и пустой обзор будет положительным, как мы узнали в главе 5 в разделе «Смещение, у-перехват и настраивание инопланетянина-молчуна». Это имеет смысл, потому что пользователи, которые оценивают фильмы негативно, как правило, оставляют отзывы. А многие из тех, кто оценивает фильмы положительно, не оставляют никаких отзывов. Остальные слова нейтральны, поэтому их вес не имеет большого значения. Рассмотрим веса некоторых слов, таких как *wonderful*, *horrible* и *the*:

- wonderful — 1,043;
- horrible — -1,075;
- the — 0,0005.

Как видите, вес слова *wonderful* положительный, вес слова *horrible* отрицательный, а вес слова *the* невелик. Это разумно: *wonderful* — это позитивное слово, *horrible* — негативное, а *the* — нейтральное.

В качестве последнего шага найдем самые положительные и отрицательные отзывы. Для этого используем модель составления прогнозов для всех фильмов. Эти прогнозы будут сохранены в новой графе под названием `predictions` с помощью следующей команды:

```
movies['prediction'] = model.predict(movies, output_type='probability')
```

Давайте найдем самые позитивные и самые негативные фильмы согласно модели. Для этого отсортируем массив следующим образом.

- **Самый положительный отзыв:**

```
movies.sort('predictions')[-1]
```

Вывод: It seems to me that a lot of people don't know that *Blade* is actually a superhero movie on par with *X-Men*... (Мне кажется, многие не знают, что «Блэйд» — это фактически супергеройский фильм наравне с «Людьми Икс»...)

- **Самый негативный отзыв:**

```
movies.sort('predictions')[0]
```

Вывод: Even duller, if possible, than the original... (Гораздо более скучный, чем оригинал...)

Эту модель можно значительно улучшить. Например, некоторые методы обработки текста, такие как удаление знаков препинания, прописных букв или стоп-слов, таких как *the*, *and*, *of*, *it*, как правило, дают лучшие результаты. Но приятно знать, что с помощью пары строк кода мы можем создать собственный классификатор анализа настроений!

Классификация по нескольким классам. Логистическая функция со многими переменными

До сих пор мы видели, как непрерывные перцептроны классифицируют два класса: радостный и грустный. Но что если классов больше? В конце главы 5 мы упоминали, что дискретному перцептрону сложно классифицировать более чем два класса. Однако такая задача по плечу логистическому классификатору.

Представьте набор данных изображений с тремя метками: «собака», «кошка» и «птица». Способ построения классификатора, который прогнозирует одну из этих трех меток для каждого изображения, заключается в создании трех классификаторов, по одному для каждой метки. Когда поступает новое изображение, мы оцениваем его с помощью каждого из трех классификаторов. Классификатор, соответствующий определенному животному, возвращает вероятность того, что изображение является соответствующим животным. Затем мы классифицируем изображение как животное из классификатора, который вернул наибольшую вероятность.

Однако это не идеальный способ, потому что классификатор возвращает дискретный ответ, такой как «собака», «кошка» или «птица». А если нам нужен классификатор, который возвращает вероятности для трех животных? Скажем, ответ может иметь вид «10 % собака, 85 % кошка и 5 % птица». Здесь на помощь приходит логистическая функция со многими переменными.